VitalFlow — Database Architecture

> Multi-tenant healthcare SaaS on Supabase Postgres 17. Tenancy, PHI handling, and auditability are first-class — every design decision below defers to HIPAA + enterprise multi-tenancy requirements before convenience.

Design principles

1. **Single Postgres, many tenants.** Shared-DB + `tenant_id` column on every tenant-scoped row. Row Level Security (RLS) is the enforcement boundary; the application layer is defense-in-depth, not the primary guard.
2. **Append-only audit.** Every mutation on PHI/financial data writes to `audit.audit_events` via trigger. Audit rows cannot be updated or deleted by any role other than `postgres`.
3. **No physical deletes for clinical data.** Soft-delete via `deleted_at`. HIPAA requires retention; physical delete is reserved for compliance requests and runs through a controlled path, not `DELETE ... FROM`.
4. **Timezone-aware timestamps everywhere.** `timestamptz`, never `timestamp`.
5. **UUIDs for every primary key.** `gen_random_uuid()` (v4). IDs never leak tenant info and are safe to expose in URLs. Business identifiers (MRN, invoice number) are separate generated columns.
6. **Enum-typed status columns.** Native Postgres enums — typos fail at write time rather than surface downstream.
7. **Idempotent, forward-only migrations.** Every migration wraps schema changes in `BEGIN` / `COMMIT`, uses `IF NOT EXISTS`, and has a matching compensating migration if it needs to be reversed.
8. **Least-privilege roles.** `authenticated` for logged-in users, `anon` locked out of PHI, `service_role` for server-side admin. No direct DB access for tenants outside PostgREST.

Schemas

Schema	Purpose
`public`	All application tables
`audit`	Append-only audit trail. No UPDATE/DELETE permissions for end-user roles.
`extensions`	Postgres extensions (pgcrypto, citext, vector, uuid-ossp, pg_trgm).

Extensions enabled

Extension	Used for
`pgcrypto`	`gen_random_uuid()`, column-level encryption helpers.
`citext`	Case-insensitive text for emails, slugs.
`pg_trgm`	Fuzzy search on patient names, drug names.
`vector`	`pgvector` for AI embeddings (semantic search on notes, knowledge base).
`btree_gin`	Mixed B-tree + GIN indexes (e.g. tenant_id + JSONB).
`pg_stat_statements`	Query observability (Supabase default).

Tenancy model

Every tenant-scoped table has:

```
tenant_id uuid not null references public.tenants(id) on delete restrict
```

RLS policy pattern (applied to every such table):

```
create policy tenant_isolation on <table>
  for all
  to authenticated
  using (tenant_id in (select public.current_user_tenant_ids()))
  with check (tenant_id in (select public.current_user_tenant_ids()));
```

The helper `public.current_user_tenant_ids()` is `SECURITY DEFINER`, queries `tenant_members` for `auth.uid()`, and is marked `STABLE` so the planner caches it per-statement.

Role checks ride on top of tenant isolation via `public.has_permission(perm text)`:

```
create policy clinical_read on encounters
  for select to authenticated
  using (
    tenant_id in (select public.current_user_tenant_ids())
    and public.has_permission('clinical:read')
  );
```

Audit strategy

- `audit.audit_events` captures `(tenant_id, actor_id, table_name, row_id, action, before jsonb, after jsonb, occurred_at, request_id, ip, user_agent)`.
- Trigger `audit.log_change()` is attached to every PHI/financial table.
- `before` and `after` are redacted copies; PHI-heavy columns are hashed rather than stored raw in the audit record (policy is defined per-table).
- `audit.audit_events` grants `INSERT` only to `authenticated` and `service_role`. `SELECT` is gated by RLS to the tenant's compliance role.

Authentication & profiles

- Supabase Auth manages `auth.users`; we do not duplicate it.
- `public.profiles` mirrors `auth.users.id` (`id uuid primary key references auth.users(id) on delete cascade`) and stores display name, avatar, locale.
- `public.tenant_members` is the M:N join between users and tenants, carrying `roles role[]` for each membership. One user can belong to many tenants; the active tenant is resolved per-request by subdomain/header.
- On user signup, a trigger inserts into `profiles`; tenant assignment is an explicit admin action (invites), not automatic.

Domains

Clinical

Table	Grain
`patients`	One row per patient per tenant. MRN unique per tenant.
`patient_contacts`	Phones, emails, addresses — multi-row.
`patient_identifiers`	External IDs (SSN hashed, insurance member IDs).
`allergies`	Allergy list — soft-delete for history.
`problems`	Problem list (active/inactive diagnoses).
`medications`	Medication list (current + historical).
`immunizations`	Vaccine history.
`encounters`	Visit records (ambulatory, ER, telehealth).
`encounter_notes`	SOAP / progress / discharge notes.
`vitals`	BP, HR, temp, SpO2, BMI etc.
`orders`	CPOE orders (lab, imaging, med, referral).
`order_results`	Lab/imaging results.

`prescriptions`	Outgoing eRx.
`attachments`	Pointers to Supabase Storage objects.
`signatures`	Clinician attestation (who/when/what).

ERP / Revenue Cycle

Table	Grain
`payers`	Insurance companies (tenant-scoped).
`patient_coverages`	Policies a patient has.
`charges`	Billable items generated from encounters.
`invoices`	Patient-facing invoices.
`invoice_lines`	Line items on an invoice.
`payments`	Received payments (patient or payer).
`claims`	Insurance claims.
`claim_lines`	Line items on a claim.
`claim_status_history`	Claim lifecycle audit (separate from audit.*).

Scheduling & inventory

Table	Grain
`appointments`	Scheduled visits.
`staff_schedules`	Clinician availability blocks.
`inventory_items`	Drugs/supplies stocked per location.
`inventory_transactions`	Stock movement (receipt, dispense, waste).

Workflow

Table	Grain
`workflow_definitions`	Workflow templates (version-pinned).
`workflow_runs`	Running instances of a definition.
`tasks`	Human-in-the-loop tasks.
`task_comments`	Collaboration thread per task.

AI

Table	Grain
`ai_requests`	Every LLM invocation (provider, model, tokens, cost).
`ai_completions`	Response text, latency, safety verdict.
`ai_embeddings`	`vector(3072)` rows, linked to source (note/document).
`ai_feedback`	Human review (thumbs up/down, free text, correction).

Monetization

Table	Grain
`subscriptions`	One active sub per tenant (Stripe subscription).
`usage_meters`	Meter definitions per plan.
`usage_events`	Raw usage events (rolled up for Stripe reporting).
`entitlements`	Per-tenant feature entitlements / plan limits.

Platform

Table	Grain
`notifications`	Outbound notification queue.
`notification_preferences`	Per-user, per-channel opt-in/out.
`integration_connections`	OAuth/creds for external systems.
`webhook_deliveries`	Incoming + outgoing webhook audit.
`feature_flags`	Flag definitions (boolean/percent/targets).
`feature_flag_overrides`	Per-tenant / per-user overrides.

Indexing strategy

- **Tenant-first composite indexes** on every list query: `(tenant_id, <sort_column> DESC)`. Postgres prunes non-matching tenants before touching the rest.
- **FK columns** always indexed (Postgres does NOT auto-index FKs).
- **Partial indexes** on `WHERE deleted_at IS NULL` for hot read paths.
- **GIN indexes** on `jsonb` columns we filter (`metadata` , `tags`).
- **pg_trgm GIN** on patient name fields for "search as you type".
- `vector` **IVFFlat** on `ai_embeddings.embedding` (lists tuned per scale).

PHI encryption

- Supabase encrypts at rest (AES-256) by default.
- Extra-sensitive columns (SSN) are **hashed with per-tenant salt** and never stored in plaintext. Exact-match lookup uses the hash; display shows last-4 via a separate unhashed column.
- File attachments go to Supabase Storage with tenant-prefixed paths and RLS-protected download URLs (signed, short-lived).

Backup & DR

- Supabase PITR (Point-in-Time Recovery) ON for production project.
- Daily logical dump of `public` and `audit` schemas to cold storage.
- Quarterly restore drill from backup → staging.

Migration sequence

#	File	Domain
01	`0001_extensions_and_helpers.sql`	Extensions, helper funcs, `audit` schema
02	`0002_tenancy_auth_audit.sql`	tenants, profiles, members, roles, audit
03	`0003_clinical_core.sql`	patients, encounters, vitals, meds, etc.
04	`0004_clinical_orders.sql`	orders, results, notes, Rx, attachments
05	`0005_erp_billing.sql`	payers, coverages, charges, invoices, claims
06	`0006_scheduling_inventory.sql`	appointments, schedules, inventory
07	`0007_workflow.sql`	workflow_definitions, runs, tasks
08	`0008_ai.sql`	ai_requests, completions, embeddings
09	`0009_monetization.sql`	subscriptions, usage, entitlements
10	`0010_platform.sql`	notifications, integrations, feature_flags

Each migration is independently applyable and leaves the database in a consistent state. Rolling back requires a matching `drop` migration.